SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

NATURAL LANGUAGE PROCESSING

ASSIGNMENT – UNIT I

Course Code: DSA0311 | CO1 | Total Marks: 30

Name: V.V.D.Nagasai Ratan Reddy	**Register Number:** 192324172
Department: AI&DS	**Date:** 05-11-2026

Question 1: Meaning and Scope of Absolute Date Expressions

An absolute date expression identifies a calendar day or a named annual occasion without relying on the time at which the speaker is talking. Examples include “March 15”, “the 22nd of November”, “15 March 2026”, and “Christmas”. These expressions can be interpreted independently of the current day, although an omitted year may require context to identify a particular occurrence.

The task is to design a finite-state automaton (FSA) that recognizes common English forms of absolute dates. The automaton reads tokens from left to right, follows labelled transitions, and accepts when the complete input follows a path to a final state. It is a surface-form recognizer, not a complete calendar reasoning system.

Deictic expressions are excluded because their meaning depends on the utterance time. “Yesterday”, “tomorrow”, “next Monday”, and “the day before yesterday” are relative expressions. The proposed machine does not include these forms. It concentrates on month-and-day patterns, day-and-month patterns, year-bearing forms, and named holidays.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Tokenization and Input Assumptions

Before an FSA processes text, the input should be tokenized. A tokenizer separates words and punctuation into a consistent sequence. For example, “March 15, 2026.” can be represented as March,

15, comma, 2026, and full stop. The full stop can be treated as sentence punctuation outside the date span.

The recognizer is case-insensitive for month names and holiday names. Thus “March 15”, “march 15”, and “MARCH 15” may follow the same path. Whitespace is treated as a token separator rather than as a meaningful word. A practical implementation should normalize repeated spaces and tabs.

The design recognizes standard English date orderings. It allows a month followed by a day, a day followed by a month, an optional article “the”, the preposition “of”, and an optional year. Abbreviations such as “Jan.” can be added to the month lexicon if the application requires them.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Lexical Classes Used in the FSA

To avoid drawing dozens of arcs, the FSA uses named lexical classes. MONTH is the set January, February, March, April, May, June, July, August, September, October, November, December. One transition labelled MONTH therefore represents twelve possible words.

DAY is a numeric day from 1 to 31. ORDINAL is an ordinal form such as 1st, 2nd, 3rd, 4th, 11th, 22nd, or 31st. YEAR is a four-digit year such as 2024, 2025, or 2026. These classes may be recognized by a lexical analyzer or a regular expression.

ARTICLE represents “the”; OF represents “of”; COMMA represents “,”; and HOLIDAY represents a recognized named annual occasion such as Christmas, Christmas Day, New Year’s Day, Valentine’s Day, Halloween, or Independence Day. The classes keep the state graph compact and readable.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: States and Accepting Conditions

The start state is q0. It represents the point before any token of the date expression has been read. From q0, the machine may enter a month-first branch, a day-first branch, or a named-holiday branch. Intermediate states represent partial expressions that are not yet complete dates.

An accepting state is reached only after the machine has recognized a complete date pattern. For example, after MONTH followed by DAY, the machine reaches an accepting state. A year may optionally follow. Likewise, a day-first sequence followed by a month reaches an accepting state.

An incomplete expression such as “March”, “the 22nd of”, or “November 2026” should not be accepted by this grammar. The acceptance condition is therefore tied to a complete path rather than to the presence of a date-related word alone.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Overall FSA Transition Design

The month-first branch begins with q0 --MONTH--> q1. The machine then reads a day using q1 --DAY--> q2. State q2 is accepting, so “March 15” is recognized. If a comma and year follow, the machine continues q2 --COMMA--> q3 and q3 --YEAR--> q4, where q4 is also accepting.

The day-first branch accepts either an ordinal or a numeric day. The optional article is handled by q0 --ARTICLE--> q5 followed by q5 --ORDINAL--> q6. Without the article, q0 may transition directly to q6 on ORDINAL. The machine then accepts a month directly or after “of”.

The holiday branch uses q0 --HOLIDAY--> qH, with qH accepting. Multiword holiday names may be represented as a single lexical token class or as a small chain of states. This is a compact FSA because the graph uses classes rather than separate arcs for every month and holiday.

FSA transition summary

q0 --MONTH--> q1 --DAY--> q2* --COMMA--> q3 --YEAR--> q4*
q0 --ARTICLE--> q5 --ORDINAL--> q6 --OF--> q7 --MONTH--> q8*
q0 --ORDINAL--> q6 --MONTH--> q8*
q0 --HOLIDAY--> qH*

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Month-First Dates: March 15

The expression “March 15” is the simplest month-first example. The first token March belongs to MONTH, so the automaton moves from q0 to q1. The token 15 belongs to DAY, so the automaton moves from q1 to q2. Since q2 is accepting, the complete expression is recognized.

The same path recognizes “January 1”, “April 30”, “December 25”, and “October 31”. The DAY class must allow one- or two-digit values. It should also validate the range 1 through 31 before the expression is accepted as a plausible calendar date.

An optional year can be included. “March 15, 2026” follows MONTH, DAY, COMMA, YEAR. If the application accepts “March 15 2026” without a comma, an additional transition from q2 to the year state can be provided. The grammar should state which punctuation style it supports.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Day-First Dates: 22nd November

The day-first pattern is common in British and international English. “22nd November” begins with an ORDINAL token and continues with a MONTH token. The FSA therefore moves from the day-first state to the accepting month state.

The article form “the 22nd of November” contains ARTICLE, ORDINAL, OF, and MONTH. The article is optional, and “of” is optional in some variants. A compact grammar can include both paths while keeping the same accepting state for a complete date.

Other examples include “1st January”, “the 3rd of April”, “15 August”, and “the 31st of October”. The machine should not accept a day expression by itself because the month is needed to complete the date under this design.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Ordinal and Numeric Day Forms

Ordinal forms express order within a month: first, second, third, fourth, and so on. In numeric spelling, suffixes such as st, nd, rd, and th are attached to the number. The FSA can treat these as members of a single ORDINAL class.

The suffix must agree with the number in ordinary English spelling. Examples include 1st, 2nd, 3rd, 4th, 11th, 12th, 13th, 21st, 22nd, and 23rd. A lexical validator can enforce these conventions before passing a token to the automaton.

Some date expressions use a cardinal number without an ordinal suffix, as in “15 November”. If these are required, the day-first branch should accept DAY as well as ORDINAL. This demonstrates how lexical classes can be extended without changing the overall FSA architecture.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Named Holidays: Christmas

Named holidays are absolute date expressions because they identify recurring calendar occasions by name. The simplest path is q0 --HOLIDAY--> qH, where qH is accepting. This recognizes “Christmas” as a complete expression.

The HOLIDAY class may contain Christmas, Christmas Day, New Year’s Day, Valentine’s Day, Halloween, and Independence Day. It may also contain jurisdiction-specific holidays, provided the system's intended coverage is clearly defined.

Some holidays move from year to year or are observed on different dates in different countries. The FSA only recognizes the written name. A separate calendar knowledge base is needed to map the holiday to a date for a specified country and year.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Multiword Holiday Expressions

A multiword holiday can be handled in two ways. First, the tokenizer may identify the entire expression as one named entity and classify it as HOLIDAY. Second, the FSA may use a short subgraph for the component words.

For example, New Year’s Day can be represented as q0 --New--> h1, h1 --Year’s--> h2, and h2 --Day--> qH. Christmas Day can be represented as q0 --Christmas--> h3 and h3 --Day--> qH. Each complete name ends at an accepting state.

The subgraph approach is useful when the system must report the exact words that form the holiday. The single-class approach is simpler when a named-entity recognizer has already identified the complete phrase.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Years and Optional Punctuation

Many dates include a year. YEAR is defined as a four-digit number within the range supported by the application. The year may follow a month-first expression after a comma, as in “March 15, 2026”, or a day-first expression without a comma, as in “15 March 2026”.

The FSA can include optional punctuation by adding a COMMA transition before YEAR. If both comma and no-comma styles are required, q2 can transition directly to an accepting year state on YEAR as well as through a comma state.

Sentence punctuation should generally be handled outside the date expression. In “Christmas is celebrated in December.” the period does not belong to the holiday name. The tokenizer can preserve the date span and leave the full stop for sentence processing.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Calendar Validation After Recognition

An FSA can recognize a string that has the correct shape but is not a valid calendar date. For example, “April 31” follows MONTH plus DAY, but April has only 30 days. “February 30” is also impossible in the Gregorian calendar.

Calendar validation should therefore follow syntactic recognition. The system maps the month name to a month number and checks the day against the number of days in that month. If a year is present, February 29 is valid only in leap years.

The Gregorian leap-year rule is: a year divisible by 400 is a leap year; a year divisible by 100 but not by 400 is not; otherwise a year divisible by 4 is a leap year. This rule belongs in a validator, not in the basic FSA.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Positive Test Cases

Positive tests verify that the automaton accepts valid expressions. Month-first examples include “March 15”, “January 1”, and “December 25, 2026”. Day-first examples include “22nd November”, “the 22nd of November”, and “15 March 2026”.

Holiday tests include “Christmas”, “Christmas Day”, and “New Year’s Day”. Each input should be tokenized, classified, and traced through the corresponding branch. The final state must be accepting for the expression to pass.

A test record can include the input, token classes, state sequence, expected result, and actual result. Keeping such records makes it easier to find missing transitions and to explain why a particular date was accepted.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Negative Test Cases and Exclusions

Negative tests ensure that the FSA does not accept incomplete expressions. “March” stops after the month state and should not be accepted. “The 22nd of” stops before a month and should also be rejected. “November 2026” does not match the specified month-day pattern.

Deictic expressions such as “yesterday”, “tomorrow”, “next week”, and “the day before yesterday” must be rejected because they depend on the current date. They are not absolute dates under the question’s definition.

Some syntactically shaped expressions may be rejected by the calendar validator rather than by the FSA. “February 30” is an example. It is important to distinguish syntactic rejection from semantic calendar validation.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Ambiguity and Contextual Interpretation

Some words can have more than one grammatical role. “May” can be a month name or a modal verb, and “March” can be a month or a verb. A local FSA may recognize a candidate date without determining the intended meaning in the full sentence.

Context helps resolve ambiguity. “The appointment is on May 5” strongly indicates a date. “May I ask a question?” does not. A larger NLP pipeline can use part-of-speech tagging, named-entity recognition, and sentence context to confirm the date interpretation.

The FSA should therefore be treated as a candidate recognizer. It identifies sequences with date-like structure. Contextual analysis and calendar validation then determine whether the candidate is genuinely a date and what normalized value it represents.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Implementation Using a Transition Table

A simple implementation stores transitions in a table. Each row contains a current state, an input class, and a next state. The recognizer starts at q0, classifies each token, looks up the matching transition, and moves to the next state.

If no transition exists for the current state and input class, the candidate is rejected. If all tokens are consumed and the final state is accepting, the candidate is accepted. A practical implementation also records the beginning and ending token positions of the match.

Lexical dictionaries should be maintained separately from the transition table. Adding a new holiday or a month abbreviation should not require redesigning the state graph. This separation makes the recognizer easier to test and maintain.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Evaluation: Precision and Recall

The recognizer can be evaluated using precision and recall. Precision is the number of correctly recognized date expressions divided by all expressions recognized by the system. Recall is the number of correctly recognized dates divided by all true date expressions in the test set.

False positives occur when the system accepts a non-date expression, such as a month name used as a verb. False negatives occur when a valid date expression is missed, perhaps because the grammar does not include an abbreviation or an alternative word order.

A balanced test corpus should include month-first dates, day-first dates, named holidays, year-bearing forms, punctuation variations, and negative deictic expressions. These cases provide a realistic assessment of the FSA.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Extensions and Limitations

The proposed FSA covers common English date forms, but it does not attempt to recognize every possible expression. It excludes date ranges such as “March 15–18”, abbreviated forms unless added to the MONTH class, and complex phrases such as “the fifteenth day of March”.

The grammar can be extended with weekday-plus-date expressions, such as “Monday, March 15”, or with month abbreviations such as “Sept. 22”. Each extension requires additional lexical classes or transitions and should be accompanied by positive and negative tests.

Relative dates remain outside the scope. If an application later needs them, they should be handled by a separate deictic-date component that uses the current date as a reference. Keeping absolute and relative date recognition separate avoids confusion.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Complete FSA Summary and Conclusion

The FSA recognizes three main patterns: MONTH followed by DAY with an optional year; an optional ARTICLE followed by DAY or ORDINAL, optionally OF, and MONTH; and a named HOLIDAY expression. It uses word classes so that the graph remains compact.

The machine accepts examples such as “March 15”, “the 22nd of November”, “15 March 2026”, and “Christmas”. It excludes deictic expressions such as “yesterday” and “the day before yesterday”. Calendar validation is performed after recognition.

In conclusion, finite-state automata provide a clear method for recognizing structured date expressions. The proposed design combines a small set of states with reusable lexical classes and can be expanded with additional date formats when required.

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 1: Worked FSA Examples and Final Review

Consider “March 15”. The token March belongs to MONTH and the token 15 belongs to DAY. The state sequence is q0, q1, q2, and q2 is accepting. Consider “the 22nd of November”. The token classes are ARTICLE, ORDINAL, OF, MONTH, and the machine follows the day-first branch to its accepting state.

Consider “Christmas”. The complete expression belongs to HOLIDAY, so the machine moves directly from q0 to qH. In contrast, “yesterday” is not a member of any accepted class and is rejected. “February 30” may match the surface pattern but is rejected by calendar validation.

The design demonstrates the central benefit of finite-state recognition: a compact set of states and class-labelled transitions can recognize many date expressions. It remains understandable, testable, and extensible while clearly excluding relative expressions.

Final acceptance patterns

MONTH DAY [COMMA YEAR] | [ARTICLE] ORDINAL [OF] MONTH | HOLIDAY

A useful way to test this part of the design is to classify each input token before tracing the state path. The classifier should identify month names, day numbers, ordinal forms, years, articles, prepositions, punctuation, and holiday names. Once tokens have been assigned to classes, the automaton no longer needs to know every spelling on every transition. This separation between lexical classification and state movement keeps the FSA compact.

The recognizer should return not only an accept or reject result but also the span of text that was recognized. For example, in a sentence such as “The event is on March 15, 2026.” the date span ends after the year, while the final full stop belongs to the surrounding sentence. Preserving the span makes it easier for a later component to normalize the date, validate it, or display it to the user.

Question 2: Regular Expression Notation and Word Definition

This solution uses Perl-compatible regular-expression notation (PCRE-style). The notation is suitable because it supports character classes, repetition, anchors, capturing groups, backreferences, lookaheads, and word-boundary assertions. The patterns are written for English alphabetic words using A through Z.

The question defines a word as an alphabetic string separated from other words by whitespace, punctuation, line breaks, or similar separators. The class [A-Za-z] represents an English letter. The expression [A-Za-z]+ therefore matches a non-empty alphabetic string.

Important operators are ^ for the beginning of a line or input, $ for the end, + for one or more repetitions, * for zero or more repetitions, ? for an optional item, and {m,n} for bounded repetition. Parentheses group expressions and can capture text for later use.

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (a): All Alphabetic Strings

The language in part (a) contains every non-empty string consisting entirely of alphabetic characters. The PCRE expression is ^[A-Za-z]+$. The anchors require the entire input to match, and the plus sign requires at least one letter.

The character class [A-Za-z] accepts uppercase and lowercase English letters. Examples accepted include “Hello”, “NLP”, “abc”, “R”, and “Computer”. The expression rejects an empty string because the plus quantifier does not permit zero characters.

Strings containing digits, spaces, punctuation, or symbols are rejected. Examples include “hello2”, “data science”, “well-known”, and “abc!”. If the task were to find words inside a larger document, word boundaries could be used instead of anchoring the whole input.

Pattern

^[A-Za-z]+$

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (a): Detailed Testing and Variants

A good test set includes the shortest accepted string, a string containing only uppercase letters, a string containing only lowercase letters, and a mixed-case string. It should also include strings with digits and punctuation to confirm that the character class is restrictive.

Test cases: “A” should match; “z” should match; “AbCd” should match; the empty string should not match; “abc123” should not match; “hello!” should not match; and “two words” should not match.

If Unicode alphabetic strings are required, a Unicode-aware PCRE engine can use \p{L}+ instead of [A-Za-z]+. That change allows letters from many writing systems. The expression in this assignment uses English letters because the examples and course context are English.

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (b): Lowercase Strings Ending in b

The required language contains only lowercase alphabetic characters, and the final character must be b. The expression is ^[a-z]*b$. The star permits zero or more lowercase letters before the final b, while the anchors constrain the complete string.

Accepted examples include “b”, “ab”, “cab”, “alphabetb”, and “zzb”. The shortest accepted string is the single character b. The expression rejects “B” because uppercase letters are not permitted and rejects “abc” because the final character is not b.

If at least one lowercase letter must appear before the final b, use ^[a-z]+b$. This modified expression rejects the one-character string b. The original expression is the direct interpretation of all lowercase strings ending in b.

Pattern

^[a-z]*b$

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (b): Boundary and Case Considerations

The expression is case-sensitive as written. A capital B at the end does not match. If a case-insensitive language were intended, the i modifier could be added, but that would no longer describe lowercase-only strings.

Whitespace is not included in [a-z], so “ab ” is rejected. Punctuation is also excluded, so “ab.” is rejected. The end anchor ensures that the final character of the entire input is b rather than merely the final character of a matching substring.

These examples demonstrate why both anchors matter. Without ^, the engine could match a lowercase suffix inside a larger string. Without $, the engine could match a prefix that ends in b even if more characters follow.

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (c): Consecutive Repeated Words

The language contains strings with two adjacent words that are exactly the same. Examples include “Humbert Humbert” and “the the”. A pair such as “the bug” is not repeated, and “the big bug” does not contain two immediately adjacent identical words.

A practical PCRE pattern is \b([A-Za-z]+)\s+\1\b. The first word is captured by ([A-Za-z]+). The whitespace expression \s+ requires a separator. The backreference \1 requires the next word to contain exactly the same letters.

The word-boundary assertions ensure that the captured sequence is a complete word. Thus “grotto grottoes” does not count as a repeated pair. Punctuation around the pair can remain outside the match because punctuation forms a word boundary.

Pattern

\b([A-Za-z]+)\s+\1\b

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (c): Backreferences and Examples

The backreference is the key part of the pattern. A simpler expression such as \b[A-Za-z]+\s+[A-Za-z]+\b would match any two words and would incorrectly accept “the bug”. The backreference forces the second word to equal the first.

By default, a case-sensitive pattern treats “The the” as different words. If case-insensitive equality is desired, use /\b([A-Za-z]+)\s+\1\b/i. The exact choice should be stated because case conventions affect what counts as identical.

If punctuation is permitted between repeated words, the separator can be widened to [^A-Za-z]+. The expression \b([A-Za-z]+)[^A-Za-z]+\1\b allows commas or other non-letter separators. The whitespace-only form is simpler when the words are separated by spaces.

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (d): Strings over the Alphabet a,b

The alphabet contains only the symbols a and b. Every occurrence of a must have a b immediately before it and a b immediately after it. A string containing only b symbols satisfies the condition because it contains no a symbols.

A correct expression is ^(?:b+(?:ab+)*)?$. The optional outer group permits the empty string. If the string is non-empty, b+ ensures that it begins with at least one b. Each repeated ab+ adds an a followed by one or more b symbols.

The leading b ensures that each a has a preceding b. The b+ following each a ensures that each a has a following b. Because the next repeated group begins with a only after the previous b+ has been consumed, the preceding-b condition is also preserved.

Pattern

^(?:b+(?:ab+)*)?$

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces,

tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (d): Proof by Examples

Accepted strings include the empty string, “b”, “bb”, “bab”, “babb”, “babab”, and “bbabbb”. Every a in these strings has a b immediately before and after it. Strings made entirely of b symbols are also accepted.

Rejected strings include “a”, because the a has no preceding or following b; “ab”, because the a has no preceding b; “ba”, because the a has no following b; and “baa”, because the second a is not preceded by b.

The pattern can be tested by examining each a position. For every a, inspect the character immediately to its left and immediately to its right. Both must be b. This local test is equivalent to the regular expression above.

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (e): Integer at Line Start and Word at Line End

The line must begin with an integer and end with an alphabetic word. A strict PCRE pattern is ^\d+\b.*\b[A-Za-z]+$. The beginning anchor ensures the integer is first, and the ending anchor ensures the final token is a word.

The expression \d+ matches one or more digits. The boundary after the number prevents the numeric prefix from being treated as part of a longer alphanumeric token. The middle .* allows any characters between the initial integer and the final word.

If leading spaces are permitted, use ^\s*\d+\b.*\b[A-Za-z]+\s*$. If a sentence-ending period or question mark is permitted after the final word, the pattern can be adjusted to allow optional punctuation after the word.

Pattern

^\d+\b.*\b[A-Za-z]+$

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (e): Examples and Assumptions

The line “2026 students attended the seminar” matches because it begins with an integer and ends with the word seminar. “45 participants joined today” also matches. A line such as “Students numbered 45 today” fails because the first token is not an integer.

The line “45” fails because it has no final alphabetic word. “45 students 2026” fails because the final token is numeric. “abc 45 students” fails because the line does not begin with an integer.

The middle section is intentionally permissive because the question does not constrain the words or punctuation between the first integer and final word. A stricter application could replace .* with a more specific expression for the allowed line content.

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (f): Whole Words grotto and raven

The string must contain both the complete word grotto and the complete word raven. It must not accept a longer word such as grottos merely because that word begins with the letters grotto. Word boundaries are therefore essential.

A PCRE expression using positive lookaheads is ^(?=.*\bgrotto\b)(?=.*\braven\b).*$. The first lookahead checks for the whole word grotto. The second checks for the whole word raven. The order of the two words does not matter.

The word-boundary assertion \b requires a boundary before and after each target word. Thus “grotto” matches, while “grottos” does not. The same rule applies to raven and longer strings such as ravens.

Pattern

^(?=.*\bgrotto\b)(?=.*\braven\b).*$

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (f): Alternative Pattern and Tests

An alternative expression uses two branches: (?:.*\bgrotto\b.*\braven\b.*|.*\braven\b.*\bgrotto\b.*). The first branch requires grotto before raven; the second branch requires raven before grotto.

The lookahead form is shorter and expresses the requirement that both words must occur regardless of order. The alternation form is useful for understanding how the two possible orders can be represented explicitly.

Accepted examples include “A grotto was near the raven’s nest” and “The raven flew past the grotto”. Rejected examples include “The grotto was empty”, “A raven appeared”, and “The grottos contain a raven” if the only occurrence is grottos rather than the whole word grotto.

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (g): Capturing the First Word of a Sentence

The requirement is to capture the first word of an English sentence and place it in a register. In regular-expression terminology, a register can be represented by a capturing group. The pattern should skip opening punctuation and store only the alphabetic word.

For a sentence beginning at the start of a line, use ^\s*["'“‘(\[]*([A-Za-z]+)\b . Group 1 stores the first word. Optional whitespace and opening quotation marks or brackets are skipped before the capture.

For example, the pattern captures Hello from “Hello, everyone.”, Please from “(Please take your seats.)”, and Today from ““Today is important.””. The comma, full stop, quotation mark, and parenthesis are not included in the captured word.

Pattern

^\s*["'“‘(\[]*([A-Za-z]+)\b

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Part (g): Sentences Within a Paragraph

A sentence may begin after a period, question mark, or exclamation mark on the same line. A pattern that finds the first word after a sentence boundary is (?:^|[.!?]\s+)["'“‘(\[]*([A-Za-z]+)\b . The capturing group stores the first alphabetic word.

The expression allows either the beginning of the input or a sentence-ending punctuation mark followed by whitespace. It then skips common opening punctuation and captures the first word. This supports multiple sentences in a paragraph rather than only the first line.

This is an approximation because abbreviations such as Dr. or etc. contain periods that do not always end a sentence. Ellipses, initials, quotations, and unusual punctuation may require a sentence tokenizer or additional rules.

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Consolidated Regular Expression Answer Table

The seven patterns are summarized here using PCRE notation. (a) ^[A-Za-z]+$; (b) ^[a-z]*b$; (c) \b([A-Za-z]+)\s+\1\b; (d) ^(?:b+(?:ab+)*)?$; (e) ^\d+\b.*\b[A-Za-z]+$; (f) ^(?=.*\bgrotto\b)(?=.*\braven\b).*$; and (g) ^\s*["'“‘(\[]*([A-Za-z]+)\b.

The patterns use different features for different constraints. Character classes define the allowed letters, anchors constrain the position, quantifiers specify repetition, backreferences enforce word equality, lookaheads require multiple words, and capturing groups save the first word.

The exact pattern may need adjustments for punctuation, Unicode letters, and multiline input. The assumptions should be stated alongside the pattern so that the expected matching behavior is unambiguous.

Pattern

(?:^|[.!?]\s+)["'“‘(\[]*([A-Za-z]+)\b

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a

complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Testing Strategy and Common Errors

Each expression should be tested with both matching and non-matching strings. For whole-word conditions, test a target word alone and as part of a longer word. For repeated words, test identical adjacent words and two different words. For line patterns, test the first and final tokens carefully.

Common errors include forgetting the anchors, using a character class that permits spaces accidentally, omitting a backreference for repeated words, and failing to use word boundaries around grotto and raven. These errors can cause false matches.

A good regular-expression answer explains the role of every important symbol. The expression alone may be correct, but a short explanation demonstrates why it satisfies the language definition and helps identify any assumptions about punctuation or case sensitivity.

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Limitations of Regular Expressions

Regular expressions are effective for local text patterns, but they do not automatically understand sentence meaning. The first-word pattern may mistake an abbreviation period for a sentence boundary. The whole-word pattern may need Unicode-aware boundaries for languages beyond English.

Character classes such as [A-Za-z] cover only basic English letters. A Unicode property such as \p{L} is more appropriate when accented characters or other writing systems are included. The chosen engine must support the property syntax.

Regular expressions should therefore be tested in the engine in which they will run. Perl, PCRE, Python, and Java have overlapping but not identical features. This assignment states PCRE-style notation and uses common constructs supported by modern engines.

Pattern

See consolidated expressions in the answer summary.

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Conclusion for Question 2

All seven requested languages can be represented using PCRE-style regular expressions. The expressions combine character classes, repetition, anchors, word boundaries, backreferences, lookaheads, and capturing groups.

The most important principle is to match complete words when the language is defined in terms of words. Word boundaries prevent partial matches, while anchors ensure that requirements about the beginning or end of a line are enforced.

In conclusion, regular expressions provide concise formal descriptions of many text patterns. Their correctness depends on carefully defining the alphabet, token boundaries, punctuation, case sensitivity, and the intended scope of matching.

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 2: Worked Examples and Final Review of All Patterns

Part (a) accepts “Language” and rejects “Language 2”. Part (b) accepts “crab” and rejects “Crab”. Part (c) accepts “very very” and rejects “very good”. Part (d) accepts “babb” and rejects “ba”. Part (e) accepts “2026 events begin today” and rejects “Events begin today 2026”.

Part (f) accepts “A raven rested inside the grotto” and rejects “The grottos contained a raven” when there is no separate whole-word occurrence of grotto. Part (g) captures “Welcome” from “(Welcome, everyone!)” and captures the first word following a sentence boundary when the paragraph pattern is used.

These examples confirm that every expression must be tested against the exact language definition. Anchors, boundaries, and capture groups are not decorative syntax: they enforce the beginning and end constraints, distinguish whole words, and save the requested first word.

Final notation statement

Notation used throughout Question 2: Perl-compatible regular expressions (PCRE-style).

When testing this expression, it is important to distinguish a full-string match from a substring match. Anchors should be used whenever the language definition refers to the whole input or to the beginning or end of a line. Word boundaries should be used whenever a requirement refers to a complete word. These details prevent a pattern from accepting a valid-looking fragment inside a larger invalid string.

The pattern should also be tested with punctuation and whitespace because the question explicitly defines words as alphabetic strings separated by such characters. A robust test set includes spaces, tabs, commas, full stops, quotation marks, and line breaks where relevant. The chosen PCRE engine and flags should be documented so that another person can reproduce the same matching behavior.

Question 3: Introduction to Minimum Edit Distance

Minimum edit distance measures the least cost required to transform one string into another. It is widely used in Natural Language Processing for spelling correction, approximate matching, optical character recognition, speech recognition, and text evaluation. The value depends on which edit operations are allowed and how much each operation costs.

This problem compares the source word “drive” with the target words “brief” and “divers”. The task permits any version of distance. This solution uses standard Levenshtein distance, in which insertion, deletion, and substitution each cost one, while matching characters cost zero.

A smaller edit distance indicates that fewer character-level operations are needed. The metric does not directly measure pronunciation or meaning. It provides a formal and reproducible comparison based on the written character sequences.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Edit Operations: Insertion, Deletion, Substitution

An insertion adds a character to a string. For example, inserting s at the end of “diver” produces “divers”. A deletion removes a character. Deleting r from “drive” produces “dive”. A substitution replaces one character with another, such as changing d to b.

In standard Levenshtein distance, each of these operations has cost one. If a character already matches the target at the relevant position, no edit is needed and its cost is zero. The distance is the minimum total cost over all valid edit sequences.

Adjacent transposition is not a primitive operation in ordinary Levenshtein distance. Swapping two neighboring characters must be represented by a combination of insertions, deletions, or substitutions. A different metric, Damerau–Levenshtein distance, includes transposition as an operation.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Direct Comparison: drive and brief

The source is d-r-i-v-e and the target is b-r-i-e-f. Both words contain five letters. The second character r and the third character i match in the same positions. The first, fourth, and fifth positions differ.

One possible transformation is drive to brive by substituting d with b; brive to briee by substituting v with e; and briee to brief by substituting the final e with f. This sequence uses three substitutions and therefore has total cost three.

A direct edit sequence gives an upper bound, but it does not alone prove that the distance is minimal. A dynamic-programming matrix evaluates all possible prefix alignments and establishes the minimum value.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Dynamic Programming Definition

Let D(i,j) be the minimum edit distance between the first i characters of the source and the first j characters of the target. The source has length m and the target has length n. The completed matrix has m+1 rows and n+1 columns because the empty prefix is included.

The base cases are D(i,0)=i and D(0,j)=j. Transforming a prefix of length i into an empty string requires i deletions. Transforming an empty string into a prefix of length j requires j insertions.

For i and j greater than zero, D(i,j) is the minimum of deletion D(i−1,j)+1, insertion D(i,j−1)+1, and substitution or match D(i−1,j−1)+cost. The cost is zero when the two current characters are equal and one otherwise.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Matrix Construction for drive to brief

Write the source characters down the rows and the target characters across the columns. Include an empty prefix at the top and left. Initialize the first row and first column with increasing integers beginning at zero.

Fill each interior cell using the recurrence. Compare the current source and target characters. If they are equal, the diagonal candidate has no added cost. If they differ, the diagonal candidate adds one. Also calculate the deletion and insertion candidates and choose the smallest.

The final cell is D(5,5), which represents the distance between the complete words. The matrix for drive and brief gives a final value of three. This confirms that three unit-cost edits are sufficient and necessary.

Calculation note

Use insertion = 1, deletion = 1, substitution = 1, and match = 0. The bottom-right matrix cell is the minimum edit distance.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Completed Matrix: drive to brief

The completed Levenshtein matrix is shown below. Rows represent the prefixes of drive and columns represent the prefixes of brief. The first row and column represent empty prefixes. The bottom-right value is the required distance.

Matrix values:
Ø b r i e f
Ø 0 1 2 3 4 5
d 1 1 2 3 4 5
r 2 2 1 2 3 4

i 3 3 2 1 2 3
v 4 4 3 2 2 3
e 5 5 4 3 2 3

The bottom-right cell is 3. Therefore Levenshtein(drive, brief)=3. The value agrees with the explicit three-substitution sequence described earlier.

Calculation note

Use insertion = 1, deletion = 1, substitution = 1, and match = 0. The bottom-right matrix cell is the minimum edit distance.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Interpreting the drive–brief Matrix

The diagonal values become smaller when the corresponding characters match. The r and i characters match in the same order and help create a low-cost alignment through the middle of the matrix. The mismatches at the beginning and end contribute to the total cost.

The matrix considers insertions and deletions as well as substitutions. It does not assume that characters at the same index must always be aligned. This is important because an optimal alignment may include gaps.

The final value of three means that no sequence of fewer than three unit-cost operations can transform drive into brief under the standard Levenshtein model. It is a minimum, not simply the cost of one arbitrary transformation.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Comparing drive and divers

The second target is divers, containing six letters. The source drive contains five letters. The words share several letters, but the order of r and i differs. The target also contains an additional s at the end.

The source can be written as d-r-i-v-e and the target as d-i-v-e-r-s. A useful alignment preserves the sequence d-i-v-e. The r in the source is deleted, and r and s are inserted after the e in the target.

This gives a transformation using one deletion and two insertions, for a total cost of three. The dynamic-programming matrix is used to confirm that no cheaper sequence exists.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Completed Matrix: drive to divers

The matrix has six rows for the empty prefix and five source characters, and seven columns for the empty prefix and six target characters. The first row and first column are initialized with the number of insertions or deletions required.

Matrix values:
Ø d i v e r s
Ø 0 1 2 3 4 5 6
d 1 0 1 2 3 4 5
r 2 1 1 2 3 4 5
i 3 2 1 2 3 4 5
v 4 3 2 1 2 3 4
e 5 4 3 2 1 2 3

The bottom-right cell is 3. Therefore Levenshtein(drive, divers)=3. This agrees with the three-operation sequence: delete r from drive, insert r after e, and insert s at the end.

Calculation note

Use insertion = 1, deletion = 1, substitution = 1, and match = 0. The bottom-right matrix cell is the minimum edit distance.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Optimal Edit Sequence for divers

Start with drive. Delete the second character r to obtain dive. This is one deletion and costs one. The remaining characters d-i-v-e are in the same order as the first four characters of divers.

Insert r after e to obtain diver. This is one insertion and raises the total cost to two. Insert s at the end to obtain divers. This is a second insertion and raises the total cost to three.

The resulting sequence is drive → dive → diver → divers. Since the dynamic-programming matrix also gives three, the sequence is optimal. There may be other optimal alignments, but none can have a cost below three.

Calculation note

Use insertion = 1, deletion = 1, substitution = 1, and match = 0. The bottom-right matrix cell is the minimum edit distance.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Final Comparison: Which Word Is Closer?

The calculated distances are Levenshtein(drive, brief)=3 and Levenshtein(drive, divers)=3. Both targets require a minimum total of three unit-cost edits from the source word drive.

Therefore, neither brief nor divers is strictly closer under standard Levenshtein distance. They are equally close, or tied. The conclusion is based on the same operation set and the same costs for both comparisons.

Visual similarity or meaning may suggest a different judgment, but the question asks for edit distance. The correct result under the stated model is a tie at distance three.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Why Character Order Matters

Edit distance is sensitive to character order. Two words may contain many of the same letters but still require several edits if the letters appear in a different sequence. The comparison of drive and divers illustrates this point.

Ordinary Levenshtein distance does not treat a transposition as one operation. If two adjacent letters are swapped, the transformation may require a deletion and an insertion or other edits. This is why an apparently small rearrangement can increase the distance.

The metric is nevertheless consistent: it counts the minimum operations from one exact character sequence to another. The same rules apply regardless of whether the words look or sound similar to a human reader.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Alternative: Damerau–Levenshtein Distance

Damerau–Levenshtein distance extends Levenshtein distance by allowing an adjacent transposition as an additional edit operation. This is useful for common typing errors such as “teh” for “the”, where two neighboring characters have been reversed.

Under a transposition-aware model, some strings that require two edits under ordinary Levenshtein distance may require only one. This can change the relative distances between candidate words.

The problem permits any version of edit distance, but a comparison is meaningful only if the same version and costs are used for both candidates. This solution consistently uses standard Levenshtein distance to keep the calculations clear.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Weighted Edit Costs

Some applications assign different costs to different operations. A substitution between visually similar characters may be assigned a lower cost than a substitution between unrelated characters. Keyboard-neighbor errors can also receive lower costs in spelling correction.

Weighted edit distance is useful when the application has information about likely errors. However, the cost model must be defined before the calculation. Without an explicit model, the distance value cannot be interpreted consistently.

For this assignment, all insertions, deletions, and substitutions have cost one. This unit-cost assumption is the standard Levenshtein model and makes the hand calculation straightforward.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Applications in NLP

Spell-checking systems use edit distance to rank candidate corrections. When a user types a misspelled word, the system compares it with dictionary entries and can prioritize candidates with smaller distances. Contextual language models may then choose the most plausible correction.

Optical character recognition systems use edit distance to compare recognized text with reference text. Speech recognition evaluation also uses edit operations at the word level to calculate word error rate. The same idea can be applied at the character level.

Approximate search, duplicate detection, record linkage, and data cleaning also use edit distance. However, the metric measures spelling similarity rather than meaning. Words such as car and automobile are semantically related but have a relatively large character edit distance.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Complexity of the Dynamic Programming Method

For source length m and target length n, the standard dynamic-programming algorithm fills (m+1)(n+1) cells. Its time complexity is O(mn), because each cell is computed from a constant number of neighboring cells.

The full matrix requires O(mn) space. If only the final distance is required, the algorithm can retain only the previous and current rows, reducing auxiliary space to O(min(m,n)) after choosing the shorter string as the column dimension.

For short words such as drive, brief, and divers, the full matrix is small enough to calculate by hand. For long strings, a program is more efficient and less prone to arithmetic mistakes.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Checking the Calculations

A hand calculation should be checked in three ways. First, verify that the first row and column contain the correct base values. Second, verify each interior cell using the minimum of deletion, insertion, and substitution or match. Third, verify that the final cell corresponds to the complete source and target.

An explicit edit sequence provides an independent check. Three substitutions transform drive into brief. One deletion and two insertions transform drive into divers. Both sequences have cost three, matching the matrix values.

Because the recurrence explores all possible prefix alignments, the final cell is the minimum cost. The edit sequences show that this minimum is achievable. Together, these checks establish the result reliably.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Conclusion for Question 3

Minimum edit distance is a formal measure of string difference based on the least costly sequence of insertions, deletions, and substitutions. The dynamic-programming recurrence computes the minimum by comparing every pair of prefixes.

The distance from drive to brief is 3. The distance from drive to divers is also 3. Therefore, under standard unit-cost Levenshtein distance, drive is equally close to brief and divers.

The result depends on the chosen edit model. A transposition-aware or weighted metric may produce different values. For the standard model used here, the two candidate words are tied at distance three.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and

diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Worked Example and Comparison Table

The first comparison, drive to brief, has distance three. One optimal sequence uses three substitutions: drive becomes brive, then briee, then brief. The matrix confirms that the minimum value is three.

The second comparison, drive to divers, also has distance three. Delete r from drive to obtain dive, insert r after e to obtain diver, and insert s to obtain divers. The sequence has one deletion and two insertions.

Comparison table:
Source | Target | Minimum edits | Distance
drive | brief | 3 | 3
drive | divers | 3 | 3
The two target words are tied under unit-cost Levenshtein distance.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.

Question 3: Final Verification and Overall Conclusion

The final answer follows from both the dynamic-programming matrices and explicit edit sequences. For each pair, an edit sequence of cost three demonstrates that a cost of three is achievable. The matrix calculation establishes that no lower-cost sequence exists.

Levenshtein(drive, brief)=3 and Levenshtein(drive, divers)=3. Thus drive is not closer to one target than the other under the selected metric. It is equally close to brief and divers.

This assignment has explained the FSA for absolute dates, seven PCRE regular expressions, and the minimum edit-distance calculation. Each solution states its assumptions, provides examples, and distinguishes formal recognition from later semantic interpretation.

Final numerical answer

Levenshtein(drive, brief) = 3; Levenshtein(drive, divers) = 3. Therefore, the two target words are equally close to drive.

A good manual calculation records the source and target prefixes clearly and applies the same recurrence at every cell. For each cell, write down the deletion candidate, insertion candidate, and diagonal candidate before selecting the minimum. This prevents mistakes caused by guessing an alignment from the visual appearance of the two words.

The final distance should be supported by an explicit edit sequence whose total cost equals the matrix result. The sequence proves that the reported cost can be achieved, while the dynamic-programming matrix establishes that no cheaper sequence exists. Using both checks gives a complete and convincing hand-calculated answer.